

ICLR 2022: A Neural Tangent Kernel for Infinite Tree Ensembles

Kanoh and Sugiyama derive the Tree NTK, the first kernel-theoretic account of soft decision-tree ensembles, and use it to explain why parameter sharing is free, why deep trees degenerate, and why harder splits help.

On tabular data, gradient-boosted and neural decision trees remain the models to beat. One increasingly popular variant is the soft tree ensemble: instead of the hard, greedy splits of a classic decision tree, every splitting rule is made differentiable, so the whole forest can be trained end-to-end with gradient descent, exactly like a neural network. These models perform well, and practitioners have accumulated a toolbox of tricks to train them: sharing parameters across nodes, tuning how sharp the splits are, over-parameterizing the ensemble. What has been missing is theory. Almost nothing explained why soft tree ensembles train the way they do, when they generalize, or why those tricks work.

For neural networks, the tool that filled a similar gap was the Neural Tangent Kernel (NTK). In the limit of infinitely many hidden units, a wide network trained by gradient descent behaves like kernel regression with a fixed, closed-form kernel, and that observation has explained training dynamics and generalization for the MLP, the CNN, and many other architectures. Each architecture induces its own distinct kernel. But no one had derived an NTK for tree models, so the soft-tree side of the tabular world stayed theoretically dark.

This paper, presented at ICLR 2022 by Ryuichi Kanoh and Mahito Sugiyama (National Institute of Informatics, NII, and SOKENDAI), closes that gap. It introduces the Tree Neural Tangent Kernel (TNTK): the kernel induced by an ensemble of infinitely many soft trees. From a single closed-form kernel, the authors derive global training convergence, prove that the parameter sharing used by oblivious-tree models costs nothing in this limit, show why very deep soft trees stop generalizing, and explain why sharper splits produce a more useful kernel.

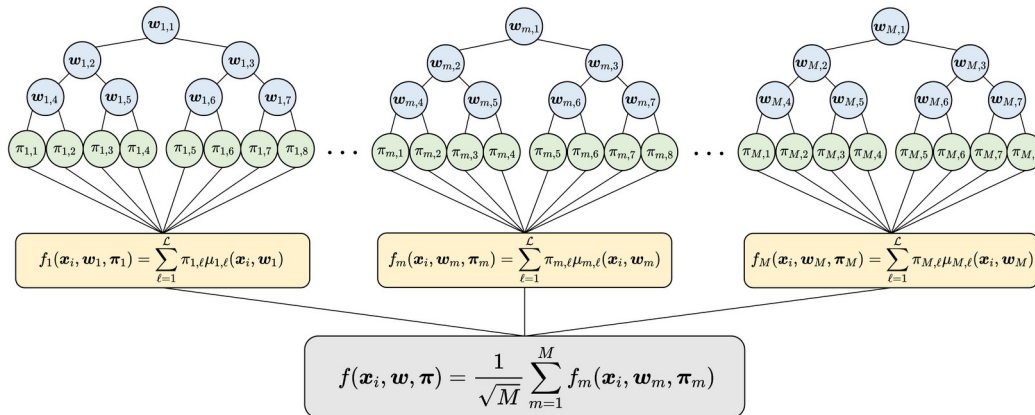


Figure 1. An ensemble of M soft trees. Each perfect binary tree routes an input through differentiable internal nodes (blue, with weight vectors w) down to leaf values (green, the response π). Every tree outputs a weighted sum over its leaves, and the ensemble averages the M tree outputs with a $1/\sqrt{M}$ scaling. The TNTK studies the limit $M \rightarrow \infty$.

From soft trees to a fixed kernel

A soft tree replaces each hard split with a smooth decision. At every internal node, instead of sending an input strictly left or right, the tree computes a probability of going each way using a scaled error function — essentially a smooth sigmoid whose steepness is set by a scaling parameter α . Small α gives a gentle, almost linear split; as α grows the split sharpens, and in the limit $\alpha \rightarrow \infty$ it recovers the hard split of an ordinary decision tree. Because every operation is differentiable, the internal weights and the leaf values can all be learned jointly by gradient descent.

The authors take an ensemble of M perfect binary soft trees of depth d , initialize the parameters randomly, and let M go to infinity. Their central result is that the ensemble's neural tangent kernel converges in probability to a deterministic kernel that no longer depends on the random initialization. The TNTK has a clean, interpretable structure: one contribution from the internal splitting nodes and one from the leaves, combined in closed form. Crucially, the split expectations have analytic expressions, so the whole kernel is available as a formula rather than a simulation. And because that formula is not a depth-by-depth recursion, computing the TNTK costs the same no matter how deep the trees are — a point that will matter later.

The formula folds together two ingredients evaluated on a pair of inputs: T , the probability that both inputs are routed the same way at a node, and its derivative in the input similarity. Written compactly, the depth- d kernel is $2^d \cdot d \cdot \Sigma \cdot T^{(d-1)} \cdot \dot{T} + (2T)^d$, where Σ is the inner product of the two inputs. The first term is the internal-node contribution, the second the leaf contribution; both are fixed once α and d are chosen.

Why training is guaranteed to converge

The reason a fixed kernel is such a useful object is that it turns a hard question about gradient descent into an easy one about linear algebra. The authors show that during training the TNTK stays essentially constant, so learning an infinite soft-tree ensemble reduces to kernel regression with the TNTK. Because the kernel is positive definite, that regression has a unique solution and gradient descent converges to it globally — there is no getting stuck in a bad local minimum. Before trusting this limit, though, it is worth checking that a finite ensemble actually approaches it.

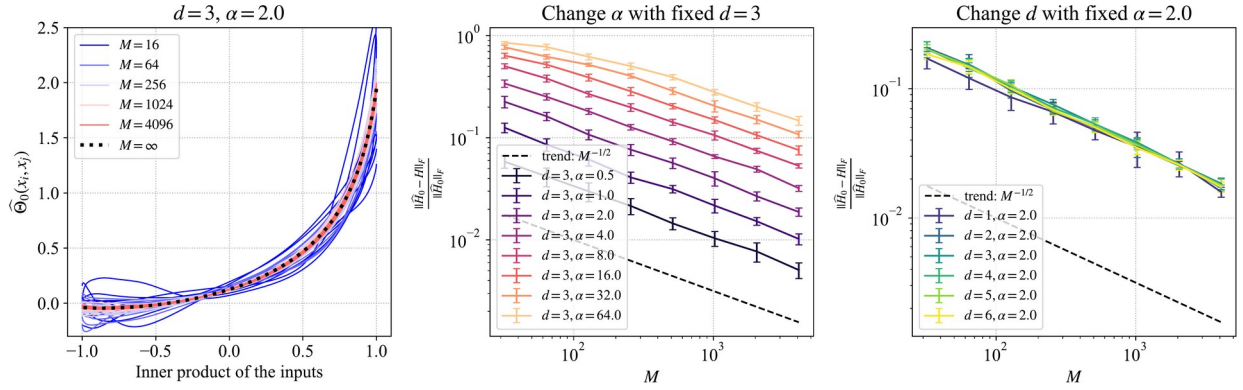


Figure 2. The finite-ensemble kernel approaches the closed-form TNTK. Left: for two inputs whose angle is swept, the empirical kernel (colored) tightens onto the analytic limit (dotted black) as the number of trees M grows from 16 to 4096. Center and right: the relative error between the empirical and limiting kernels falls at the expected $M^{-1/2}$ rate across split-hardness α and depth d .

The convergence is not just asymptotic hand-waving. As the ensemble grows, the empirical kernel tightens onto the analytic TNTK at the $M^{-1/2}$ rate the theory predicts, and it does so consistently as the split hardness α and the tree depth d are varied. With the limiting kernel validated, the kernel-regression picture of training can be tested directly.

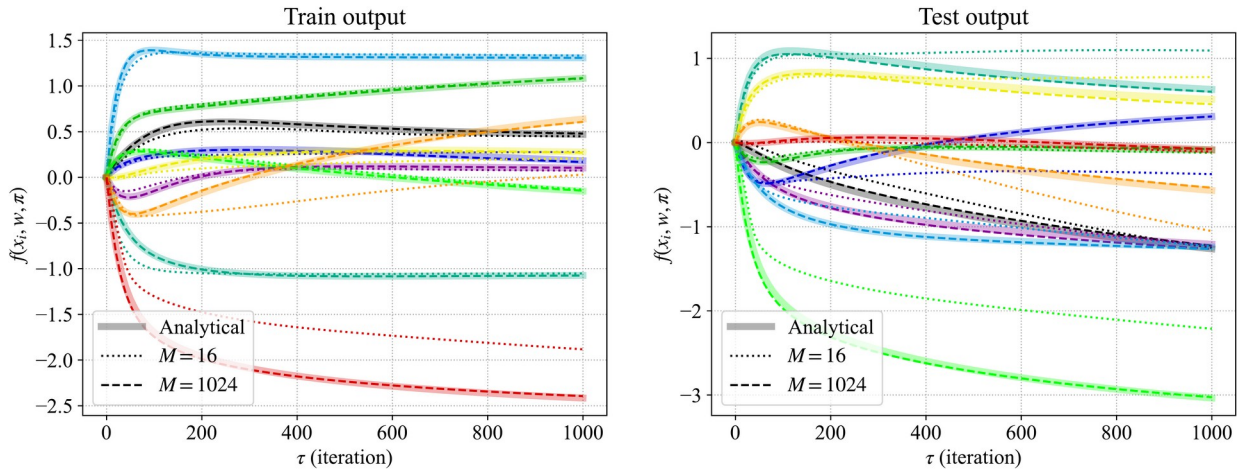


Figure 3. Training dynamics match the kernel-regression prediction. Each color is one train (left) or test (right) point; the thick analytical curve is the trajectory the limiting TNTK predicts, and the dashed lines are actual soft-tree ensembles ($M = 1024$) trained by gradient descent. Even a modest ensemble ($M = 16$, dotted) tracks the prediction, and $M = 1024$ follows it closely.

When the authors train real soft-tree ensembles and overlay the output trajectory the TNTK predicts, the two agree — closely for $M = 1024$, and recognizably even for $M = 16$. The infinite-width theory is describing the behavior of finite, trainable models, not just an idealized limit.

Oblivious trees come for free

A popular design choice in models like NODE is the oblivious tree: within a given depth, all internal nodes share the same splitting rule, which drastically cuts the number of parameters

and makes the tree easy to vectorize. It is natural to worry that this constraint sacrifices expressive power. The TNTK gives a clean answer.

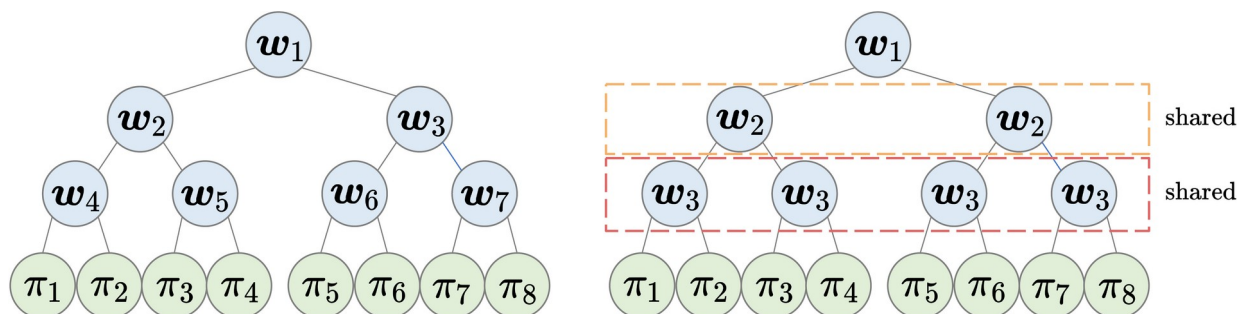


Figure 4. Normal tree (left) versus oblivious tree (right). In the oblivious tree, the splitting rule is shared across all nodes at the same depth (the boxed w -vectors are identical), while the leaf responses π can still differ. The paper proves both induce the same TNTK in the infinite limit.

The authors prove that an ensemble of oblivious soft trees converges to exactly the same TNTK as an ensemble of ordinary soft trees. In the infinite limit, sharing splitting rules within a depth costs nothing in kernel terms — the very architectural constraint that makes oblivious trees efficient turns out to be free. That is a reassuring theoretical footing for a design many practitioners already use.

Sharper splits help; deeper trees degenerate

Because the TNTK is available in closed form, one can simply plot it and read off how the two knobs — split hardness α and depth d — reshape the kernel. This is where the theory starts to explain empirical folklore.

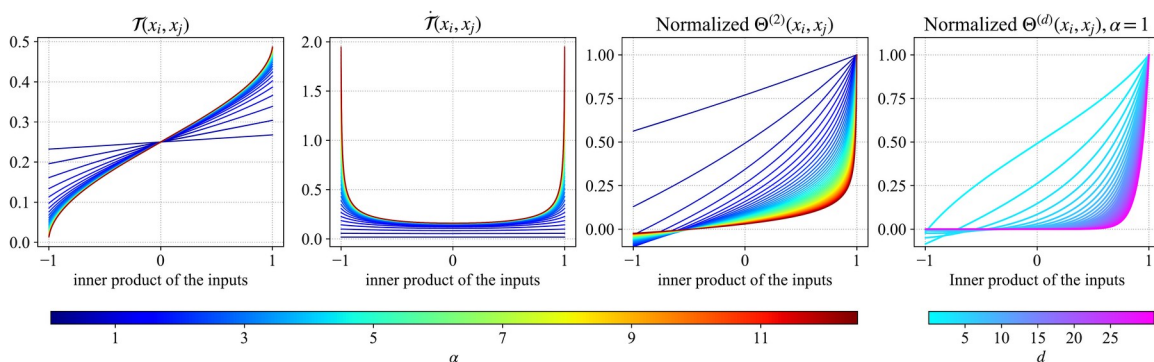


Figure 5. How α and depth shape the kernel, as a function of the inner product between two inputs. Left three panels (colored by α): as splits sharpen, the kernel bends away from a straight line into a more expressive, nonlinear shape. Right panel (colored by depth d): as trees get deeper, the kernel flattens toward zero everywhere except at perfect similarity — the degeneracy that predicts poor generalization.

Two readings come straight off these curves. First, when α is small the kernel is nearly linear, so the model can barely express nonlinear structure; as α grows the kernel bends into a richer shape. This matches the intuition that a near-linear soft split is too gentle to be useful. Second, and less obviously, as depth increases the kernel degenerates: it collapses toward a constant, responding only when two inputs are almost identical. A kernel that treats nearly all

pairs of inputs as equally dissimilar cannot generalize well, which is the paper's explanation for why very deep soft trees underperform.

Does it work? Kernel regression on 90 datasets

Theory in hand, the authors test the TNTK as a practical kernel. They run kernel regression with the TNTK on 90 real-world datasets, tuning depth from $d = 1$ to 29, and compare against the kernel induced by an infinitely wide MLP and against the classic RBF kernel.

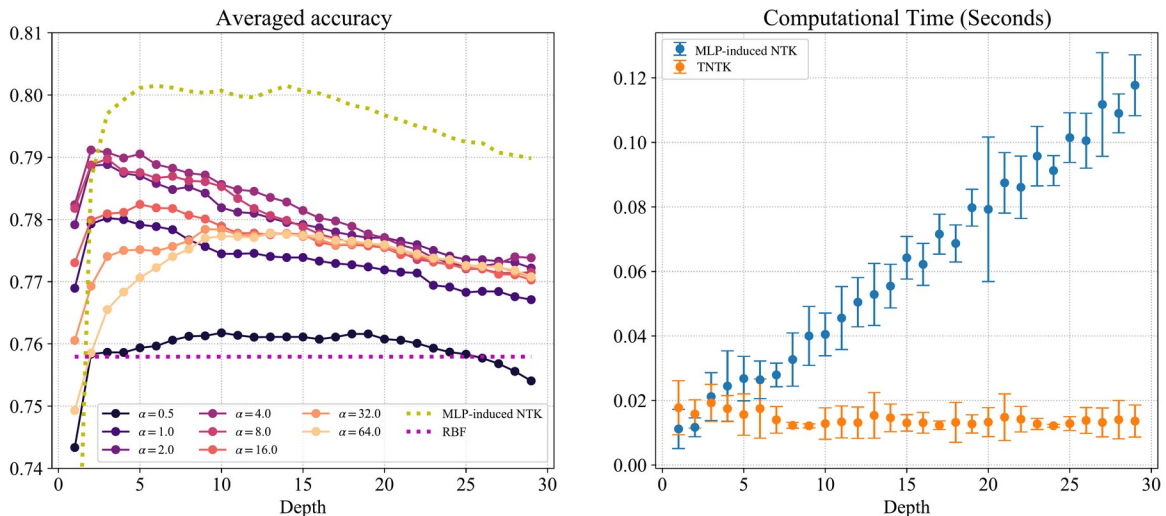


Figure 6. Left: averaged accuracy over 90 datasets versus tree depth, for a range of split-hardness values α , with the MLP-induced NTK and RBF baselines shown for reference. Accuracy first rises then falls with depth — exactly the degeneracy the kernel shape predicts. Right: kernel-computation time versus depth. The TNTK (orange) stays flat while the MLP-induced NTK (blue) grows linearly with depth.

The depth sweep reproduces the predicted degeneracy directly: averaged accuracy climbs for the first few levels and then declines as trees get deeper, for every setting of α . The timing panel makes the closed-form payoff concrete — because the TNTK does not recurse in depth, its cost is essentially flat, whereas the MLP-induced NTK grows linearly with depth. On the accuracy comparison, the MLP-induced kernel is stronger on average, but the tree kernel is not merely a curiosity: it wins outright on more than 30% of the individual datasets, exactly the cases where a tree's inductive bias is the better fit.

The split-hardness sweep quantifies the "sharper is better" story. The table below reports, for each α , the fraction of the 90 datasets on which the TNTK beats the MLP-induced NTK. The win rate climbs from 13.6% at the near-linear $\alpha = 0.5$ to a peak of 34.9% at $\alpha = 32$, and the whole hard-split range sits comfortably above the 11.8% managed by the RBF kernel.

Table 1. TNTK win rate against the MLP-induced NTK across the 90 datasets, as a function of split hardness α (depth tuned from $d = 1$ to 29). The RBF kernel is shown for reference.

α	0.5	1.0	2.0	4.0	8.0	16.0	32.0	64.0	RBF
Win rate (%)	13.6	18.8	22.2	28.6	32.5	31.6	34.9	27.2	11.8

Takeaways

The lasting contribution is conceptual: the NTK framework, previously a tool for neural networks, extends cleanly to tree ensembles. The resulting Tree NTK explains a range of soft-tree behavior from one theory — training converges globally, oblivious parameter sharing is free in the infinite limit, deep trees degenerate into a near-constant kernel, and sharper splits yield a more expressive one. Along the way it delivers a usable, depth-free kernel that is competitive with, and on more than 30% of datasets better than, the MLP-induced NTK, at a fraction of the computational cost as depth grows.

The honest boundaries are worth stating. The analysis lives in the infinite-ensemble limit and covers perfect binary trees; the empirical study is kernel regression on tabular datasets rather than large-scale deep learning, and on average the MLP-induced kernel still edges out the tree kernel. Even so, the TNTK gives soft tree ensembles something they lacked — a principled account of why they train and generalize as they do, and a concrete guide to the tricks that make them work.